

Guía de Laboratorio 1

Gestión de Tareas y Semáforos en FreeRTOS

alaprovitta@unc.edu.ar

Marzo 2026

Objetivo General

Dominar la creación, parametrización y control de flujo de tareas concurrentes, junto con la sincronización básica mediante semáforos utilizando la API nativa de FreeRTOS.

Recursos Necesarios

- **Hardware:** Microcontrolador STM32, 4 LEDs (LED1 a LED4) y 1 Pulsador.
- **Software:** STM32CubeIDE.
- **Librerías permitidas:** FreeRTOS.h, task.h y semphr.h.

Desafíos Prácticos

Desafío 1: Concurrencia Básica

Objetivo: Comprender la creación de tareas y la transición entre estados de ejecución.
Comportamiento: Los 4 LEDs deben parpadear de forma completamente independiente, cada uno a una frecuencia distinta (ej. 200ms, 400ms, 600ms y 800ms). Las 4 tareas deben tener la misma prioridad y NO esta permitido usar ningun servicio bloqueante de FreeRTOS.

Servicios requeridos: `xTaskCreate()`: Para instanciar 4 tareas distintas. Pueden hacer uso del servicio `HAL_Delay(ms)` de la libreria `STM32_HAL_Cube`.

Análisis: Todas las tareas se ejecutan? Hay concurrencia entre tareas? Porque?. Los tiempos de conmutación de los leds son precisos? (Medirlos de ser posible utilizando periodos de conmutacion mas pequeños: 20ms, 40ms, 60ms y 80ms)

Desafío 2: Instanciación Múltiple y Paso de Parámetros

Objetivo: Evitar la duplicación de código mediante el uso del puntero `pvParameters` y comprender la transición entre estados de ejecución con bloqueo.

Comportamiento: Crear una **única** función de tarea llamada `vTareaParpadeo`. Se deben crear 4 instancias de esta tarea (4 llamadas a `xTaskCreate`). Cada instancia debe controlar un LED diferente y parpadear a una frecuencia distinta (ej: 200ms, 400ms, 600ms, 800ms). Se debe utilizar servicios bloqueantes para generar la temporización de los leds.

Servicios requeridos: `xTaskCreate` (para crear 4 instancias de la misma función parametrizada), estructura (`struct`) para empaquetar puerto, pin y retardo. `vTaskDelay()`: Para ceder el control del procesador (`Yield`) mientras se espera el próximo cambio de estado del LED.

Análisis: Que diferencias hay en las transiciones de tareas respecto al caso del Desafío 1? (De ser posible medir con fines comparativos utilizando los mismos periodos de conmutación del caso anterior).

Desafío 3: Polling vs. Bloqueo Eficiente

Objetivo: Aprender a leer entradas físicas sin acaparar el tiempo de CPU (evitar la inanición de tareas de menor prioridad).

Comportamiento: El LED1 parpadea constantemente a 500ms mediante una tarea no bloqueante (que nunca se bloquea) de baja prioridad. El LED2 debe encenderse solo mientras el PULSADOR1 esté presionado. La lectura del botón debe hacerse en una tarea de mayor prioridad mediante "Polling" (sondeo continuo en el bucle de la tarea), no por interrupción. El tiempo de respuesta de LED 2 no puede superar los 10 ms.

Servicios requeridos: `vTaskDelay()` con un tiempo de espera acorde en la tarea del botón. Si se omite este delay, el LED1 dejará de parpadear (demostración práctica de Starvation).

Análisis: Probar con diferentes tiempos para `vTaskDelay()`. Que efecto se observa a medida que ese tiempo aumenta? Que tiempo satisface el deadline impuesto en la consigna?

Desafío 4: Periodicidad Absoluta vs. Relativa (El problema de la Deriva)

Objetivo: Comprender la diferencia entre retardos relativos (`vTaskDelay`) y retardos absolutos periódicos (`xTaskDelayUntil`), y cómo prevenir la deriva de tiempo (*Time Drift*) generada por el tiempo de ejecución del propio código y el algoritmo de *Scheduling*.

Comportamiento:

- Crear dos tareas (Tarea A y Tarea B) con la misma prioridad. Ambas deben intentar hacer parpadear un LED a exactamente cada 500ms. La Tarea A controla el LED1 y la Tarea B el LED2.
- Dentro del bucle infinito de ambas tareas, y **antes** de llamar a la función de retardo, se debe introducir una carga de procesamiento simulada (utilizar un retardo bloqueante del HAL como `HAL_Delay(100)`) que consuma aproximadamente 100 ms de CPU.
- La Tarea A debe utilizar `vTaskDelay(pdMS_TO_TICKS(?))` para su retardo.
- La Tarea B debe utilizar `xTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(?))` para su retardo.

Servicios requeridos: `vTaskDelay()`, `xTaskDelayUntil()`, `xTaskGetTickCount()` (para inicializar la variable de tiempo absoluto).

Análisis: Intuitivamente determinar el tiempo (?) argumento de los servicios `xTaskDelay` para cumplir con los tiempos impuestos en la consigna. Ambos argumentos son iguales? Que efecto se observa si lo fueran? Porque?

Desafío 5: Control de Prioridades Dinámico

Objetivo: Observar cómo el planificador de FreeRTOS favorece siempre a la tarea de mayor prioridad (*Preemptive Scheduling*).

Comportamiento:

- Tarea A (Prioridad 1): Parpadea LED1 cada 300ms.
- Tarea B (Prioridad 1): Parpadea LED2 cada 300ms.
- Al presionar el Pulsador, la Tarea A debe subir su propia prioridad en 1 por encima de la prioridad actual de la Tarea B, durante solo 3 segundos (haciendo que el LED1 se encienda y el LED2 se congele durante este tiempo). Luego se debe reestablecer su prioridad original.

Servicios requeridos: `vTaskPrioritySet()`, `uxTaskPriorityGet()`.

Análisis: Que ocurre cuando la prioridad de la tarea A se reestablece? Que servicio se utilizo para hacer el conteo de los 3 segundos? Se logró el comportamiento descrito en el enunciado en el primer intento? De no ser así, porque?

Desafío 6: Sincronización Unidireccional (Deferred Interrupt Processing)

Objetivo: Reemplazar el *polling* por un mecanismo de espera pasiva.

Comportamiento: Una tarea permanece bloqueada sin consumir tiempo de CPU. Cuando se presiona el Pulsador (detectado por interrupción EXTI), se libera un semáforo. La tarea despierta, conmuta el LED3 y vuelve a bloquearse.

Servicios requeridos: `xSemaphoreCreateBinary()`, `xSemaphoreTake()`, `xSemaphoreGiveFromISR()`.

Análisis: Proponer una forma de visualizar dinamicamente (algo cambia todo el tiempo) que la tarea en cuestion efectivamente no esta consumiendo tiempo del procesador. No esta permitido crear nuevas tareas.

Desafío 7: Secuenciamiento de Tareas (Task Synchronization)

Objetivo: Utilizar semáforos para establecer un orden de ejecución estricto.

Comportamiento: Crear 3 tareas, cada una controla un LED (LED1, LED2, LED3). Las tareas deben encender sus LEDs en orden (1 → 2 → 3) y apagar el anterior. La Tarea 1 le da el "permiso" a la 2, la 2 a la 3, y la 3 vuelve a la 1.

Servicios requeridos: Múltiples `xSemaphoreBinary` actuando como señales de habilitación.

Análisis: Que se debe cambiar para alterar el orden de la secuencia? Es posible implementar una solución a este problema escribiendo una **única** función?

Desafío 8: Arbitraje de Recursos Compartidos (Mutex)

Objetivo: Comprender el acceso exclusivo a recursos compartidos.

Comportamiento: Dos tareas de igual prioridad que no se bloquean por tiempo intentan escribir en los LEDs de forma concurrente. La Tarea A hace una secuencia de encendido rápido (1-2-3-4). La Tarea B hace un parpadeo total 4 veces por secuencia (todos ON, todos OFF). Ambas tareas deben competir por el uso de los leds mediante un semáforo

tipo Mutex; la que lo obtiene debe terminar su secuencia completa antes de soltarlo, evitando que los efectos visuales se corrompan.

Servicios requeridos: `xSemaphoreCreateMutex()`, `xSemaphoreTake()`, `xSemaphoreGive()`.

Análisis: Se debe usar servicios bloqueante (`vTaskDelay()`) o no-bloqueante (`HAL_Delay()`) para contar los tiempos asociados a una secuencia? Porque? Reescriba el ejercicio utilizando el otro servicio de Delay al propuesto originalmente y compare su comportamiento.

Desafío 9: Suspensión y Reanudación de Tareas

Objetivo: Controlar el estado de ejecución de una tarea externa.

Comportamiento: La Tarea A genera un patrón de luces en los 4 LEDs. La Tarea B (monitoreo de botón) suspende la ejecución de la Tarea A con la primera pulsación (congelando los LEDs en el estado en que quedaron) y la reanuda con la segunda pulsación.

Servicios requeridos: `vTaskSuspend()`, `vTaskResume()`.

Análisis: Que consecuencias tiene para el sistema que el estado del pulsador se determine por Polling o por interrupción? Implementar la que ud. considere la solución que genera el mejor tiempo de respuesta? Esta seguro?

Desafío 10: Autodestrucción y Gestión de Memoria

Objetivo: Comprender la liberación de recursos y el ciclo de vida final de una tarea.

Comportamiento: Al iniciar el sistema, una tarea ejecuta una animación de inicio (*Light Show*) durante 10 segundos. Una vez terminada, la tarea debe encender el LED4 de forma permanente y eliminarse a sí misma para liberar la memoria de su Pila (*Stack*), permitiendo que el sistema libere carga de memoria.

Servicios requeridos: `vTaskDelete(NULL)`.

Análisis: Hay alguna forma de exteriorizar (mediante algún evento visible) que esa tarea efectivamente se ha eliminado y no se encuentra mas disponible para su ejecución?